```
"""
RecallBot — AI memory for team chats.
Upload WhatsApp/Slack exports, index them, and ask questions with cited answers.
"""

from __future__ import annotations

import json
import os
import re
import sqlite3
import time
import uuid
from datetime import datetime, timedelta
from io import BytesIO
from typing import Generator

import chromadb
import streamlit as st
import tiktoken
from dateutil import parser as dateutil_parser
from fastembed import TextEmbedding
from fpdf import FPDF
from openai import OpenAI

# ─── Constants ──────────────────────────────────────────────────────────────

CHROMA_PATH = "./chroma_db"
DB_PATH = "recallbot.db"
CHUNK_SIZE = 500
CHUNK_OVERLAP = 100
EMBED_BATCH = 128
TOP_K = 7
SIMILARITY_THRESHOLD = 0.3
MODEL = "gpt-4o-mini"
EMBED_MODEL_NAME = "BAAI/bge-small-en-v1.5"

SAMPLE_MESSAGES = [
    {"date": "12 Oct 2024", "sender": "Sara", "text": "We've decided to go with the new
onboarding flow starting November 1st."},
    {"date": "12 Oct 2024", "sender": "Alex", "text": "Action item: Alex to update the landing
page copy by Oct 20th."},
    {"date": "13 Oct 2024", "sender": "Jordan", "text": "Should we integrate Stripe or Paddle
for payments? Still unresolved."},
    {"date": "14 Oct 2024", "sender": "Sara", "text": "Key decision: The API rate limit will be
1000 req/min for free tier."},
    {"date": "15 Oct 2024", "sender": "Alex", "text": "Reminder: team sync every Monday at 10
AM. Jordan to send calendar invite."},
```

```
]

# ─── CSS ──────────────────────────────────────────────────────────────────────

DARK_CSS = """
<style>
@import url('https://fonts.googleapis.com/css2?
family=Inter:wght@300;400;500;600;700&family=JetBrains+Mono:wght@400;500&display=swap');

:root {
    --base: #0A0A0B;
    --surface: #111114;
    --surface2: #16161A;
    --accent: #6366F1;
    --accent-hover: #4F52D9;
    --text: #E5E7EB;
    --muted: #9CA3AF;
    --border: #1F2937;
    --success: #10B981;
    --danger: #EF4444;
    --radius: 10px;
    --radius-sm: 6px;
}

* { box-sizing: border-box; }

html, body, [data-testid="stApp"] {
    background-color: var(--base) !important;
    color: var(--text) !important;
    font-family: 'Inter', sans-serif !important;
}

/* Hide Streamlit chrome */
#MainMenu, footer, header { visibility: hidden; }
[data-testid="stDecoration"] { display: none; }

/* Sidebar */
[data-testid="stSidebar"] {
    background-color: var(--surface) !important;
    border-right: 1px solid var(--border) !important;
}
[data-testid="stSidebar"] * { color: var(--text) !important; }

/* Input fields */
input, textarea, [data-baseweb="input"] input {
    background: var(--surface2) !important;
    border: 1px solid var(--border) !important;
    color: var(--text) !important;
    border-radius: var(--radius-sm) !important;
    font-family: 'Inter', sans-serif !important;
```

```
}

/* Buttons */
.stButton > button {
    background: var(--accent) !important;
    color: #fff !important;
    border: none !important;
    border-radius: var(--radius-sm) !important;
    min-height: 44px !important;
    font-family: 'Inter', sans-serif !important;
    font-weight: 500 !important;
    font-size: 0.875rem !important;
    padding: 0 18px !important;
    transition: background 0.2s ease, transform 0.1s ease !important;
    cursor: pointer !important;
}
.stButton > button:hover {
    background: var(--accent-hover) !important;
    transform: translateY(-1px) !important;
}
.stButton > button:active { transform: translateY(0) !important; }

/* Secondary buttons */
.stButton.secondary > button {
    background: var(--surface2) !important;
    border: 1px solid var(--border) !important;
    color: var(--text) !important;
}
.stButton.secondary > button:hover {
    background: var(--border) !important;
}

/* Chat messages */
[data-testid="stChatMessage"] {
    background: var(--surface) !important;
    border: 1px solid var(--border) !important;
    border-radius: var(--radius) !important;
    margin-bottom: 12px !important;
    animation: fadeIn 0.3s ease;
}
[data-testid="stChatMessageContent"] * { color: var(--text) !important; }

@keyframes fadeIn {
    from { opacity: 0; transform: translateY(6px); }
    to   { opacity: 1; transform: translateY(0); }
}

/* Chat input */
[data-testid="stChatInput"] {
    background: var(--surface) !important;
```

```
    border: 1px solid var(--border) !important;
    border-radius: var(--radius) !important;
}
[data-testid="stChatInput"] textarea {
    background: transparent !important;
    border: none !important;
    color: var(--text) !important;
}

/* Cards */
.rb-card {
    background: var(--surface);
    border: 1px solid var(--border);
    border-radius: var(--radius);
    padding: 16px;
    margin-bottom: 12px;
    transition: border-color 0.2s;
}
.rb-card:hover { border-color: var(--accent); }

/* Source chip */
.rb-source-chip {
    display: inline-block;
    background: var(--surface2);
    border: 1px solid var(--border);
    border-radius: 20px;
    padding: 3px 12px;
    font-family: 'JetBrains Mono', monospace;
    font-size: 0.72rem;
    color: var(--muted);
    margin: 3px 3px 3px 0;
    cursor: pointer;
    transition: border-color 0.2s, color 0.2s;
}
.rb-source-chip:hover { border-color: var(--accent); color: var(--accent); }

/* Progress bar shimmer */
.shimmer-container {
    width: 100%;
    height: 6px;
    background: var(--surface2);
    border-radius: 3px;
    overflow: hidden;
    margin: 12px 0;
}
.shimmer-bar {
    height: 100%;
    border-radius: 3px;
    background: linear-gradient(90deg, var(--accent) 0%, #818CF8 50%, var(--accent) 100%);
    background-size: 200% 100%;
```

```
    animation: shimmer 1.5s infinite;
}
@keyframes shimmer {
    0% { background-position: 200% center; }
    100% { background-position: -200% center; }
}

/* Label / heading */
.rb-label {
    font-size: 0.7rem;
    font-weight: 600;
    letter-spacing: 0.08em;
    text-transform: uppercase;
    color: var(--muted);
    margin-bottom: 8px;
}

.rb-title {
    font-size: 1.35rem;
    font-weight: 700;
    color: var(--text);
    letter-spacing: -0.02em;
}

.rb-subtitle {
    font-size: 0.85rem;
    color: var(--muted);
    margin-top: 4px;
}

/* Highlighted term */
mark {
    background: rgba(99,102,241,0.3) !important;
    color: var(--text) !important;
    border-radius: 2px;
    padding: 0 2px;
}

/* Divider */
.rb-divider {
    border: none;
    border-top: 1px solid var(--border);
    margin: 16px 0;
}

/* Expander */
[data-testid="stExpander"] {
    background: var(--surface) !important;
    border: 1px solid var(--border) !important;
    border-radius: var(--radius) !important;
```

```
}

/* Selectbox */
[data-baseweb="select"] {
    background: var(--surface2) !important;
    border-color: var(--border) !important;
}

/* File uploader */
[data-testid="stFileUploader"] {
    background: var(--surface2) !important;
    border: 1.5px dashed var(--border) !important;
    border-radius: var(--radius) !important;
    color: var(--muted) !important;
}

/* Scrollbar */
::-webkit-scrollbar { width: 5px; height: 5px; }
::-webkit-scrollbar-track { background: var(--base); }
::-webkit-scrollbar-thumb { background: var(--border); border-radius: 3px; }

/* Mobile responsive */
@media (max-width: 768px) {
    .rb-right-panel { display: none !important; }
    [data-testid="stSidebar"] { width: 100% !important; }
}

/* Success/danger badges */
.rb-badge-success { color: var(--success); font-size: 0.8rem; }
.rb-badge-danger  { color: var(--danger);  font-size: 0.8rem; }

/* Recent questions list */
.rb-recent-q {
    font-size: 0.82rem;
    color: var(--muted);
    padding: 6px 10px;
    border-radius: var(--radius-sm);
    cursor: pointer;
    transition: background 0.15s;
    white-space: nowrap;
    overflow: hidden;
    text-overflow: ellipsis;
}
.rb-recent-q:hover { background: var(--surface2); color: var(--text); }

.stTextInput > div > div > input {
    min-height: 44px !important;
}
</style>
"""
```

```
# ─── Database helpers ──────────────────────────────────────────────────────────

def init_db() -> None:
    """Create SQLite tables for feedback and question history."""
    with sqlite3.connect(DB_PATH) as con:
        con.execute("""
            CREATE TABLE IF NOT EXISTS feedback (
                id TEXT PRIMARY KEY,
                question TEXT,
                answer TEXT,
                thumbs INTEGER,
                workspace TEXT,
                created_at TEXT
            )
        """)
        con.execute("""
            CREATE TABLE IF NOT EXISTS questions (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                question TEXT,
                workspace TEXT,
                created_at TEXT
            )
        """)
        con.commit()


def save_question(question: str, workspace: str) -> None:
    with sqlite3.connect(DB_PATH) as con:
        con.execute(
            "INSERT INTO questions (question, workspace, created_at) VALUES (?,?,?)",
            (question, workspace, datetime.utcnow().isoformat()),
        )
        con.commit()


def get_recent_questions(workspace: str, limit: int = 10) -> list[str]:
    with sqlite3.connect(DB_PATH) as con:
        rows = con.execute(
            "SELECT question FROM questions WHERE workspace=? ORDER BY id DESC LIMIT ?",
            (workspace, limit),
        ).fetchall()
    return [r[0] for r in rows]


def save_feedback(msg_id: str, question: str, answer: str, thumbs: int, workspace: str) -> None:
    with sqlite3.connect(DB_PATH) as con:
        con.execute(
            "INSERT OR REPLACE INTO feedback (id,question,answer,thumbs,workspace,created_at)
VALUES (?,?,?,?,?,?)",
```

```
            (msg_id, question, answer, thumbs, workspace, datetime.utcnow().isoformat()),
        )
        con.commit()


# ─── Cached resources ──────────────────────────────────────────────────────────

@st.cache_resource
def get_chroma_client() -> chromadb.Client:
    """Return a persistent ChromaDB client."""
    return chromadb.PersistentClient(path=CHROMA_PATH)


@st.cache_resource
def get_embedding_model() -> TextEmbedding:
    """Load fastembed BAAI/bge-small-en-v1.5."""
    return TextEmbedding(model_name=EMBED_MODEL_NAME)


def get_openai_client() -> OpenAI:
    """Return an OpenAI client using secrets."""
    key = st.secrets.get("OPENAI_API_KEY", os.environ.get("OPENAI_API_KEY", ""))
    if not key:
        st.error("OpenAI API key not found. Add OPENAI_API_KEY to st.secrets or environment.")
        st.stop()
    return OpenAI(api_key=key)


# ─── Parsing ──────────────────────────────────────────────────────────────────

def detect_format(content: str) -> str:
    """Detect whether the file is WhatsApp or Slack JSON."""
    stripped = content.strip()
    if stripped.startswith("[") or stripped.startswith("{"):
        try:
            json.loads(stripped)
            return "slack"
        except Exception:
            pass
    return "whatsapp"


def parse_whatsapp(content: str) -> list[dict]:
    """Parse WhatsApp export: [dd/mm/yy, hh:mm] Sender: Message."""
    pattern = re.compile(
        r"\[(\d{1,2}/\d{1,2}/\d{2,4}),?\s+(\d{1,2}:\d{2}(?::\d{2})?(?:\s?[AP]M)?)\]\s+
([^:]+):\s+(.*)"
    )
    messages = []
    current = None
```

```
    for line in content.splitlines():
        m = pattern.match(line.strip())
        if m:
            if current:
                messages.append(current)
            date_str, time_str, sender, text = m.groups()
            try:
                dt = dateutil_parser.parse(f"{date_str} {time_str}", dayfirst=True)
                date_fmt = dt.strftime("%d %b %Y")
            except Exception:
                date_fmt = date_str
            current = {"date": date_fmt, "sender": sender.strip(), "text": text.strip()}
        elif current and line.strip():
            current["text"] += " " + line.strip()
    if current:
        messages.append(current)
    return messages


def parse_slack(content: str) -> list[dict]:
    """Parse Slack JSON export."""
    try:
        data = json.loads(content)
        if isinstance(data, dict):
            # channel export (dict of lists)
            msgs = []
            for v in data.values():
                if isinstance(v, list):
                    msgs.extend(v)
        elif isinstance(data, list):
            msgs = data
        else:
            return []
        messages = []
        for m in msgs:
            if not isinstance(m, dict):
                continue
            text = m.get("text", "").strip()
            sender = (
                m.get("user_profile", {}).get("display_name")
                or m.get("username")
                or m.get("user", "Unknown")
            )
            ts = m.get("ts", "")
            try:
                dt = datetime.fromtimestamp(float(ts))
                date_fmt = dt.strftime("%d %b %Y")
            except Exception:
                date_fmt = "Unknown"
            if text:
```

```
                messages.append({"date": date_fmt, "sender": sender, "text": text})
        return messages
    except Exception:
        return []


# ─── Chunking ──────────────────────────────────────────────────────────────────

def chunk_messages(messages: list[dict], chunk_size: int = CHUNK_SIZE, overlap: int =
CHUNK_OVERLAP) -> list[dict]:
    """
    Convert messages to token-chunked documents with metadata.
    Returns list of {text, date, sender, chunk_id}.
    """
    enc = tiktoken.get_encoding("cl100k_base")
    chunks = []
    token_buf: list[int] = []
    meta_buf: list[dict] = []

    def flush(toks: list[int], metas: list[dict]) -> dict:
        text = enc.decode(toks)
        # use the first message's metadata for the chunk
        m = metas[0] if metas else {}
        return {
            "text": text,
            "date": m.get("date", ""),
            "sender": m.get("sender", ""),
            "chunk_id": str(uuid.uuid4()),
        }

    for msg in messages:
        msg_text = f"[{msg['date']}] {msg['sender']}: {msg['text']}"
        toks = enc.encode(msg_text)
        for tok in toks:
            token_buf.append(tok)
            meta_buf.append(msg)
            if len(token_buf) >= chunk_size:
                chunks.append(flush(token_buf, meta_buf))
                # keep overlap
                token_buf = token_buf[-overlap:]
                meta_buf = meta_buf[-overlap:]

    if token_buf:
        chunks.append(flush(token_buf, meta_buf))

    return chunks


# ─── Embedding & Indexing ──────────────────────────────────────────────────────
```

```
def embed_texts(texts: list[str]) -> list[list[float]]:
    """Embed a list of texts using fastembed."""
    model = get_embedding_model()
    embeddings = list(model.embed(texts))
    return [e.tolist() for e in embeddings]


def get_or_create_collection(workspace: str) -> chromadb.Collection:
    """Get or create a ChromaDB collection for the workspace."""
    client = get_chroma_client()
    safe_name = re.sub(r"[^a-zA-Z0-9_-]", "_", workspace)[:60] or "default"
    return client.get_or_create_collection(
        name=safe_name,
        metadata={"hnsw:space": "cosine"},
    )


def index_messages(messages: list[dict], workspace: str, progress_placeholder) -> dict:
    """
    Chunk, embed, and store messages in ChromaDB.
    Returns stats dict.
    """
    start = time.time()
    n_messages = len(messages)

    chunks = chunk_messages(messages)
    n_chunks = len(chunks)

    collection = get_or_create_collection(workspace)

    # Embed in batches
    texts = [c["text"] for c in chunks]
    ids = [c["chunk_id"] for c in chunks]
    metadatas = [{"date": c["date"], "sender": c["sender"]} for c in chunks]

    done = 0
    for batch_start in range(0, n_chunks, EMBED_BATCH):
        batch_end = min(batch_start + EMBED_BATCH, n_chunks)
        batch_texts = texts[batch_start:batch_end]
        batch_ids = ids[batch_start:batch_end]
        batch_meta = metadatas[batch_start:batch_end]
        batch_emb = embed_texts(batch_texts)

        collection.upsert(
            ids=batch_ids,
            embeddings=batch_emb,
            documents=batch_texts,
            metadatas=batch_meta,
        )
        done = batch_end
```

```
        progress_placeholder.markdown(
            f"<div class='shimmer-container'><div class='shimmer-bar' style='width:
{int(done/n_chunks*100)}%'></div></div>"
            f"<span style='color:var(--muted);font-size:0.82rem'>Indexing {n_messages:,}
messages... {done:,}/{n_chunks:,} chunks</span>",
            unsafe_allow_html=True,
        )

    elapsed = round(time.time() - start, 1)
    return {"messages": n_messages, "chunks": n_chunks, "seconds": elapsed}


# ─── RAG Pipeline ──────────────────────────────────────────────────────────────

def retrieve_chunks(query: str, workspace: str) -> list[dict]:
    """Embed query, query ChromaDB, return top-K results."""
    collection = get_or_create_collection(workspace)
    q_emb = embed_texts([query])[0]
    results = collection.query(
        query_embeddings=[q_emb],
        n_results=TOP_K,
        include=["documents", "metadatas", "distances"],
    )
    chunks = []
    docs = results["documents"][0] if results["documents"] else []
    metas = results["metadatas"][0] if results["metadatas"] else []
    dists = results["distances"][0] if results["distances"] else []
    for doc, meta, dist in zip(docs, metas, dists):
        similarity = 1 - dist  # cosine distance → similarity
        chunks.append({
            "text": doc,
            "date": meta.get("date", ""),
            "sender": meta.get("sender", ""),
            "similarity": round(similarity, 4),
        })
    return chunks


def build_context(chunks: list[dict]) -> str:
    lines = []
    for c in chunks:
        lines.append(f"[{c['date']}][{c['sender']}]: {c['text']}")
    return "\n\n".join(lines)


def stream_answer(query: str, chunks: list[dict]) -> Generator[str, None, None]:
    """Stream a GPT-4o-mini answer given retrieved chunks."""
    context = build_context(chunks)
    system = (
        "You are RecallBot. Answer ONLY using the [CONTEXT] provided. "
```

```
        "Cite sources inline as [Date][Sender]. "
        "At the end of your answer, add a 'Sources:' section listing each source as [Date,
Sender]. "
        "If the answer is not found in the context, say exactly: 'I couldn't find that in the
chat history.'"
    )
    user_msg = f"[CONTEXT]\n{context}\n\n[QUESTION]\n{query}"
    client = get_openai_client()
    try:
        with client.chat.completions.stream(
            model=MODEL,
            messages=[
                {"role": "system", "content": system},
                {"role": "user", "content": user_msg},
            ],
            max_tokens=900,
            temperature=0.2,
        ) as stream:
            for text in stream.text_stream:
                yield text
    except Exception as e:
        yield f"\n\n_Error communicating with OpenAI: {e}_"


# ─── Summarize week ──────────────────────────────────────────────────────────

def summarize_week(workspace: str) -> dict:
    """Retrieve last-7-days messages and summarize into JSON structure."""
    collection = get_or_create_collection(workspace)
    week_ago = (datetime.utcnow() - timedelta(days=7)).strftime("%d %b %Y")

    # Retrieve a broad sample (no date filter in ChromaDB easily; use large n_results)
    results = collection.get(limit=500, include=["documents", "metadatas"])
    docs = results.get("documents", [])
    metas = results.get("metadatas", [])

    context_lines = []
    for doc, meta in zip(docs, metas):
        context_lines.append(f"[{meta.get('date','')}][{meta.get('sender','')}]: {doc}")

    context = "\n".join(context_lines[:300])  # cap to avoid huge prompt

    client = get_openai_client()
    prompt = (
        "You are RecallBot summarizer. Given the following chat messages, extract a weekly
summary.\n"
        "Return ONLY valid JSON (no markdown) with this exact structure:\n"
        '{"key_decisions": ["...", "..."], "action_items": ["- [ ] Task @Owner by Date", ...], '
        '"unresolved_questions": [{"q": "...", "a": "..."}, ...]}\n\n'
        f"MESSAGES:\n{context}"
```

```
    )
    try:
        resp = client.chat.completions.create(
            model=MODEL,
            messages=[{"role": "user", "content": prompt}],
            max_tokens=1000,
            temperature=0.3,
        )
        raw = resp.choices[0].message.content or "{}"
        raw = raw.strip().lstrip("```json").lstrip("```").rstrip("```").strip()
        return json.loads(raw)
    except Exception as e:
        return {"error": str(e)}


# ─── PDF Export ──────────────────────────────────────────────────────────────

def export_answer_pdf(question: str, answer: str) -> bytes:
    """Generate a PDF for the given Q&A."""
    pdf = FPDF()
    pdf.add_page()
    pdf.set_font("Helvetica", "B", 16)
    pdf.set_text_color(10, 10, 11)
    pdf.cell(0, 10, "RecallBot — Answer Export", ln=True)
    pdf.set_font("Helvetica", "", 10)
    pdf.set_text_color(100, 100, 110)
    pdf.cell(0, 6, datetime.utcnow().strftime("Generated %d %b %Y %H:%M UTC"), ln=True)
    pdf.ln(4)
    pdf.set_draw_color(31, 41, 55)
    pdf.line(10, pdf.get_y(), 200, pdf.get_y())
    pdf.ln(4)
    pdf.set_font("Helvetica", "B", 12)
    pdf.set_text_color(10, 10, 11)
    pdf.multi_cell(0, 7, f"Question: {question}")
    pdf.ln(3)
    pdf.set_font("Helvetica", "", 11)
    pdf.multi_cell(0, 6, answer)
    buf = BytesIO()
    pdf.output(buf)
    return buf.getvalue()


def export_workspace_json(workspace: str) -> str:
    """Export all chunks from a workspace as JSON string."""
    collection = get_or_create_collection(workspace)
    results = collection.get(include=["documents", "metadatas"])
    data = []
    for doc, meta in zip(results.get("documents", []), results.get("metadatas", [])):
        data.append({"text": doc, **meta})
    return json.dumps(data, indent=2)
```

```
def highlight_terms(text: str, query: str) -> str:
    """Wrap query terms in <mark> tags."""
    terms = [t for t in query.split() if len(t) > 2]
    for term in terms:
        text = re.sub(re.escape(term), f"<mark>{term}</mark>", text, flags=re.IGNORECASE)
    return text


# ─── Session State Init ──────────────────────────────────────────────────────

def init_session() -> None:
    defaults = {
        "workspace": "default",
        "chat": [],           # list of {role, content, chunks, msg_id}
        "last_chunks": [],
        "index_stats": None,
        "rerun_q": None,
    }
    for k, v in defaults.items():
        if k not in st.session_state:
            st.session_state[k] = v


# ─── UI: Sidebar ─────────────────────────────────────────────────────────────

def render_sidebar() -> None:
    with st.sidebar:
        st.markdown("<div class='rb-title'>RecallBot</div>", unsafe_allow_html=True)
        st.markdown("<div class='rb-subtitle'>AI memory for team chats</div>",
unsafe_allow_html=True)
        st.markdown("<hr class='rb-divider'>", unsafe_allow_html=True)

        # Workspace management
        st.markdown("<div class='rb-label'>Workspace</div>", unsafe_allow_html=True)
        ws_input = st.text_input("Name", value=st.session_state.workspace,
label_visibility="collapsed", key="ws_input_field")
        col1, col2 = st.columns(2)
        with col1:
            if st.button("Switch", use_container_width=True):
                ws_name = ws_input.strip() or "default"
                if ws_name != st.session_state.workspace:
                    st.session_state.workspace = ws_name
                    st.session_state.chat = []
                    st.session_state.last_chunks = []
                    st.session_state.index_stats = None
                    st.rerun()
        with col2:
            if st.button("Delete", use_container_width=True):
```

```
                try:
                    client = get_chroma_client()
                    safe_name = re.sub(r"[^a-zA-Z0-9_-]", "_", st.session_state.workspace)[:60]
                    client.delete_collection(safe_name)
                    st.session_state.chat = []
                    st.session_state.index_stats = None
                    st.success("Workspace deleted.")
                except Exception as e:
                    st.error(f"Could not delete: {e}")

        # Export workspace
        export_json = export_workspace_json(st.session_state.workspace)
        st.download_button(
            "Export Workspace JSON",
            data=export_json,
            file_name=f"{st.session_state.workspace}_export.json",
            mime="application/json",
            use_container_width=True,
        )

        st.markdown("<hr class='rb-divider'>", unsafe_allow_html=True)

        # Upload
        st.markdown("<div class='rb-label'>Upload Chat Export</div>", unsafe_allow_html=True)
        uploaded = st.file_uploader("Drop .txt or .json", type=["txt", "json"],
label_visibility="collapsed")

        use_sample = st.button("Use Sample Data", use_container_width=True)

        progress_ph = st.empty()

        if use_sample:
            stats = index_messages(SAMPLE_MESSAGES, st.session_state.workspace, progress_ph)
            st.session_state.index_stats = stats
            st.rerun()

        if uploaded is not None:
            content = uploaded.read().decode("utf-8", errors="replace")
            fmt = detect_format(content)
            if fmt == "slack":
                messages = parse_slack(content)
            else:
                messages = parse_whatsapp(content)

            if messages:
                stats = index_messages(messages, st.session_state.workspace, progress_ph)
                st.session_state.index_stats = stats
                st.rerun()
            else:
                st.warning("No messages parsed. Check file format.")
```

```
        if st.session_state.index_stats:
            s = st.session_state.index_stats
            st.markdown(
                f"<div class='rb-badge-success'>"
                f"✓ {s['messages']:,} messages, {s['chunks']:,} chunks, indexed in
{s['seconds']}s"
                f"</div>",
                unsafe_allow_html=True,
            )

        st.markdown("<hr class='rb-divider'>", unsafe_allow_html=True)

        # Recent questions
        st.markdown("<div class='rb-label'>Recent Questions</div>", unsafe_allow_html=True)
        recents = get_recent_questions(st.session_state.workspace)
        for q in recents:
            if st.button(q[:55] + ("…" if len(q) > 55 else ""), key=f"rq_{q[:20]}",
use_container_width=True):
                st.session_state.rerun_q = q
                st.rerun()


# ─── UI: Main Chat ──────────────────────────────────────────────────────────

def render_chat() -> None:
    # Header row
    hcol1, hcol2 = st.columns([3, 1])
    with hcol1:
        st.markdown(
            f"<div class='rb-label'>Workspace: <span style='color:var(--accent)'>
{st.session_state.workspace}</span></div>",
            unsafe_allow_html=True,
        )
    with hcol2:
        if st.button("Summarize Week", use_container_width=True):
            with st.spinner("Summarizing last 7 days…"):
                summary = summarize_week(st.session_state.workspace)
            if "error" in summary:
                st.error(summary["error"])
            else:
                _render_summary(summary)

    # Chat history
    for msg in st.session_state.chat:
        with st.chat_message(msg["role"]):
            st.markdown(msg["content"])
            if msg["role"] == "assistant":
                _render_feedback_buttons(msg)
```

```
    # Handle rerun from recent question click
    if st.session_state.rerun_q:
        q = st.session_state.rerun_q
        st.session_state.rerun_q = None
        _process_query(q)
        st.rerun()

    # Chat input
    if prompt := st.chat_input("Ask about your chat history…"):
        _process_query(prompt)
        st.rerun()


def _process_query(query: str) -> None:
    """Run RAG pipeline and add messages to chat history."""
    st.session_state.chat.append({"role": "user", "content": query, "chunks": [], "msg_id":
str(uuid.uuid4())})
    save_question(query, st.session_state.workspace)

    chunks = retrieve_chunks(query, st.session_state.workspace)
    st.session_state.last_chunks = chunks

    # Check similarity threshold
    if not chunks or (chunks and chunks[0]["similarity"] < SIMILARITY_THRESHOLD):
        answer = "I couldn't find that in the chat history."
        st.session_state.chat.append({
            "role": "assistant",
            "content": answer,
            "chunks": [],
            "msg_id": str(uuid.uuid4()),
        })
        return

    # Stream answer
    answer_parts: list[str] = []
    with st.chat_message("assistant"):
        placeholder = st.empty()
        for token in stream_answer(query, chunks):
            answer_parts.append(token)
            placeholder.markdown("".join(answer_parts) + "▌")
        full_answer = "".join(answer_parts)
        placeholder.markdown(full_answer)

    msg_id = str(uuid.uuid4())
    st.session_state.chat.append({
        "role": "assistant",
        "content": full_answer,
        "chunks": chunks,
        "msg_id": msg_id,
        "question": query,
```

```
    })


def _render_feedback_buttons(msg: dict) -> None:
    """Render thumbs up/down feedback and PDF export."""
    msg_id = msg.get("msg_id", "")
    question = msg.get("question", "")
    content = msg.get("content", "")

    cols = st.columns([1, 1, 1, 5])
    with cols[0]:
        if st.button("👍", key=f"up_{msg_id}"):
            save_feedback(msg_id, question, content, 1, st.session_state.workspace)
            st.toast("Thanks for the feedback!")
    with cols[1]:
        if st.button("👎", key=f"dn_{msg_id}"):
            save_feedback(msg_id, question, content, -1, st.session_state.workspace)
            st.toast("Thanks for the feedback!")
    with cols[2]:
        pdf_bytes = export_answer_pdf(question, content)
        st.download_button(
            "PDF",
            data=pdf_bytes,
            file_name="recallbot_answer.pdf",
            mime="application/pdf",
            key=f"pdf_{msg_id}",
        )


def _render_summary(summary: dict) -> None:
    """Render weekly summary in an expander."""
    with st.expander("Weekly Summary", expanded=True):
        st.markdown("**Key Decisions**")
        for d in summary.get("key_decisions", []):
            st.markdown(f"- {d}")
        st.markdown("**Action Items**")
        for a in summary.get("action_items", []):
            st.markdown(a)
        st.markdown("**Unresolved Questions**")
        for uq in summary.get("unresolved_questions", []):
            st.markdown(f"**Q:** {uq.get('q','')}")
            st.markdown(f"**A:** {uq.get('a','')}")


# ─── UI: Right Source Panel ──────────────────────────────────────────────────

def render_sources_panel() -> None:
    chunks = st.session_state.last_chunks
    query = ""
    if st.session_state.chat:
```

```
        for msg in reversed(st.session_state.chat):
            if msg["role"] == "user":
                query = msg["content"]
                break

    st.markdown("<div class='rb-label'>Sources</div>", unsafe_allow_html=True)
    if not chunks:
        st.markdown("<span style='color:var(--muted);font-size:0.82rem'>No sources yet.</span>",
unsafe_allow_html=True)
        return

    for i, c in enumerate(chunks):
        score_color = "#10B981" if c["similarity"] >= 0.5 else "#9CA3AF"
        highlighted = highlight_terms(c["text"][:200] + ("…" if len(c["text"]) > 200 else ""),
query)
        with st.expander(f"[{c['date']}] {c['sender']}  •  {c['similarity']:.2f}", expanded=(i
== 0)):
            st.markdown(
                f"<div style='font-size:0.82rem;color:var(--muted)'>"
                f"Similarity: <span style='color:{score_color}'>{c['similarity']:.3f}</span>
</div>",
                unsafe_allow_html=True,
            )
            st.markdown(
                f"<div style='font-size:0.83rem;line-height:1.6;font-family:JetBrains
Mono,monospace'>{highlighted}</div>",
                unsafe_allow_html=True,
            )
            st.markdown(
                f"<span class='rb-source-chip'>{c['date']}, {c['sender']}</span>",
                unsafe_allow_html=True,
            )


# ─── Main ──────────────────────────────────────────────────────────────────────

def main() -> None:
    st.set_page_config(
        page_title="RecallBot",
        page_icon="🔍",
        layout="wide",
        initial_sidebar_state="expanded",
    )
    st.markdown(DARK_CSS, unsafe_allow_html=True)

    init_db()
    init_session()

    # Check API key early
    if not st.secrets.get("OPENAI_API_KEY", os.environ.get("OPENAI_API_KEY", "")):
```

```
        st.error(
            "**Missing OpenAI API key.**\n\n"
            "Add `OPENAI_API_KEY` to your Streamlit secrets (`secrets.toml`) or environment
variables."
        )
        st.stop()

    render_sidebar()

    main_col, right_col = st.columns([3, 1])

    with main_col:
        render_chat()

    with right_col:
        st.markdown("<div class='rb-card rb-right-panel'>", unsafe_allow_html=True)
        render_sources_panel()
        st.markdown("</div>", unsafe_allow_html=True)


if __name__ == "__main__":
    main()
```